React Code Lab: How to Do the Lab

A student guide to the ten auto-graded React activities

The React Code Lab is a website where you practise everything from Modules 4 to 7 by **writing real React code**. It continues the two apps from class: **Movie Night** (the tutorials) and **Alex's Café** (the labs). In each activity you write one file, watch your app render as you type, submit it, and get an instant report: your score, every mistake explained, and the proper code beside yours.

Open the lab at **rezaghaeli.github.io/csis-3380-react-code-lab**. Read this guide once before you start; it takes about ten minutes.

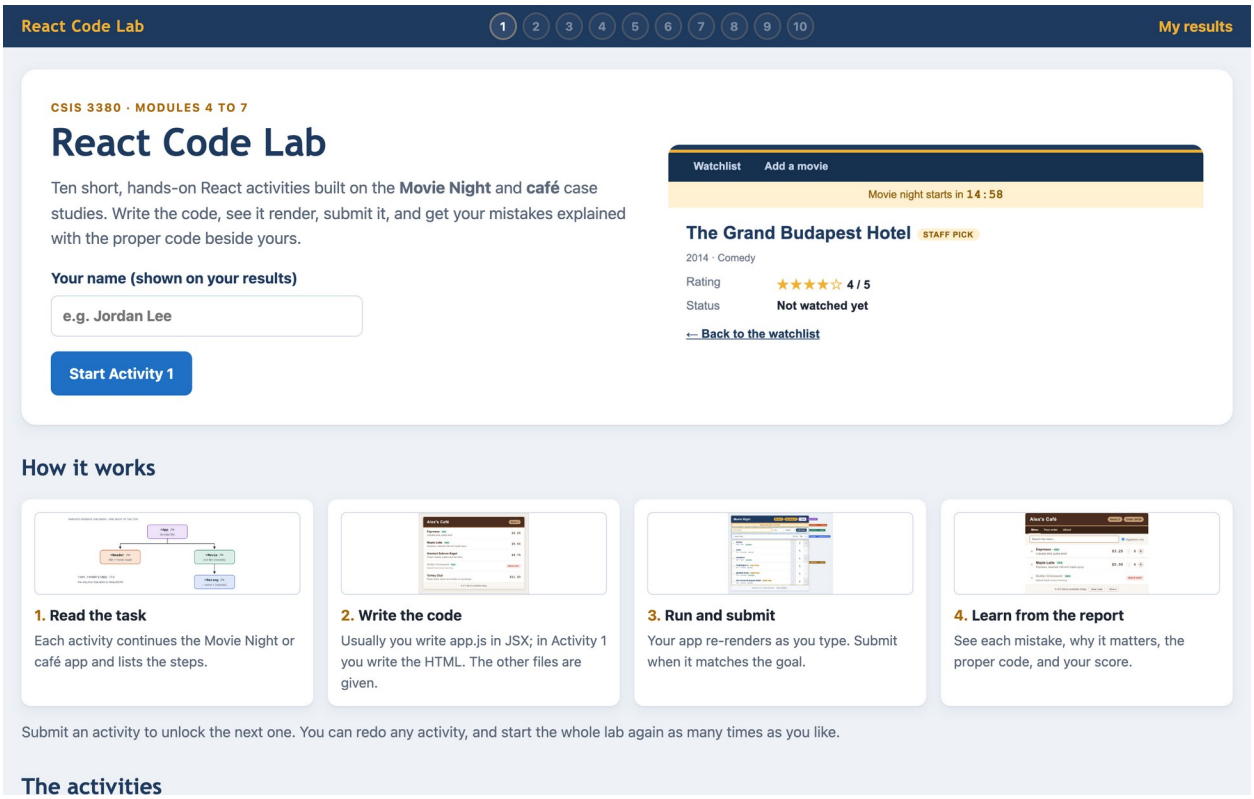


Figure 1 — the home page of the React Code Lab

CONTENTS

Section	What it covers
1	What the lab is, and the ten activities
2	What you are expected to do
3	Before you start
4	The home page
5	Inside an activity
6	Your first activity, step by step
7	Reading your report
8	Moving through the lab
9	Your final results
10	Tips for the React activities
11	Troubleshooting

---

1. What the lab is

Each activity is one small piece of the Movie Night or café app. You read a short task, write the code in an editor in the browser, and see the result straight away. When it matches the goal, you **submit** it. Submitting an activity unlocks the next one, and after Activity 10 you get an overall score for the whole lab. You can redo any activity, and start the whole lab again as many times as you like.

#	Module	You write	What you build	About
1	4	HTML	Add React to the Movie Night page (CDN scripts, root, Babel)	10 min
2	4	JSX	The header: `{ }` expressions, `&&`, and `? :`	10 min
3	4	JSX	A `Movie` component with props	10 min
4	4	JSX	The whole list with `map()` and `key`	10 min
5	5	JSX	The café order tally: a class with `this.state` and `setState`	15 min
6	5	JSX	Lifting state up: Watched buttons and a live footer	15 min
7	6	JSX	`useState` and a controlled form that adds movies	15 min
8	6	JSX	`useEffect` and `useRef`: tab title, focus, countdown cleanup	15 min
9	6	JSX	`useContext`: a light/dark theme	10 min
10	7	JSX	React Router: a details page per movie, links, a 404	15 min
			Total	**about 2 h**

You do not have to do it in one sitting. Two or three activities at a time works well.

---

2. What you are expected to do

Unless your instructor tells you otherwise:

- ☐ Complete **all ten activities, in order**.
- ☐ Write the code **yourself**. Use the steps, the handouts, and your class code. Look at the proper code only **after** you submit.
- ☐ **Read every report**: for each mistake, read *We found*, *Why*, and *The correct code*, then fix your own code.
- ☐ If an activity scores **below 80%**, fix it and **submit again** before you move on.

- ☐ Aim for an **overall score of 80% or more**.
- ☐ When you finish, **save your results page as a PDF** (Section 9) and upload it if your instructor asks for it.

> **HOW THE SCORE WORKS**
> Each activity has 5 to 8 checks worth 1 to 3 points. Your activity score is the points you earned as a percentage, and your overall score is the average of the ten activities. The lab really runs your app: it clicks your buttons, types into your inputs, and reads what React renders. It also looks at *how* you wrote some things (for example, that you used `map()` and a `key`). You can submit as often as you like; your **latest** submission counts.

---

3. Before you start

- Use a **laptop or desktop** with a current Chrome, Edge, Firefox, or Safari. Typing JSX on a phone is hard.
- You need an **internet connection**: React, ReactDOM, Babel, and React Router load from **unpkg**, a free CDN, exactly as in our CDN projects in class.
- There is nothing to install and no account or password.

> **YOUR PROGRESS IS SAVED IN YOUR BROWSER**
> The lab remembers your code and scores in the browser you use, on that computer. Use the **same computer and browser** each time, and do **not** use a private or incognito window: everything is forgotten when it closes. Clearing your browsing data also clears the lab. Nothing is sent to your instructor unless you send the PDF yourself.

---

4. The home page

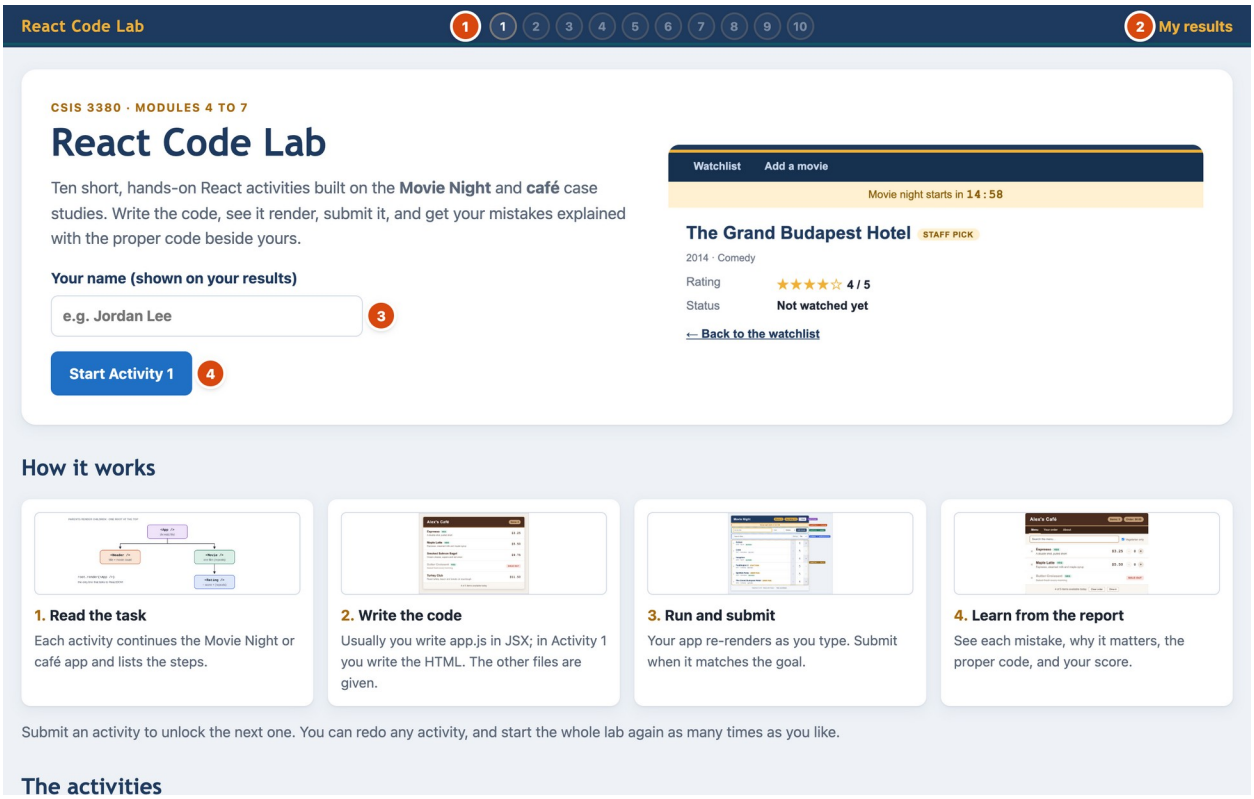


Figure 2 — the home page; the numbers match the table below

#	What it is	What to do with it
1	The activity circles	One circle per activity. Click an open one to go to it. They change colour as you finish activities (Section 8).
2	My results	Your scores so far, and the full-lab score when you finish.
3	Your name	Type your name: it appears on your results page and PDF.
4	Start / Continue	Opens the next activity you have not submitted yet.

Further down, **The activities** lists all ten. Only the next one is **Open**; the others show a padlock until you submit the one before.

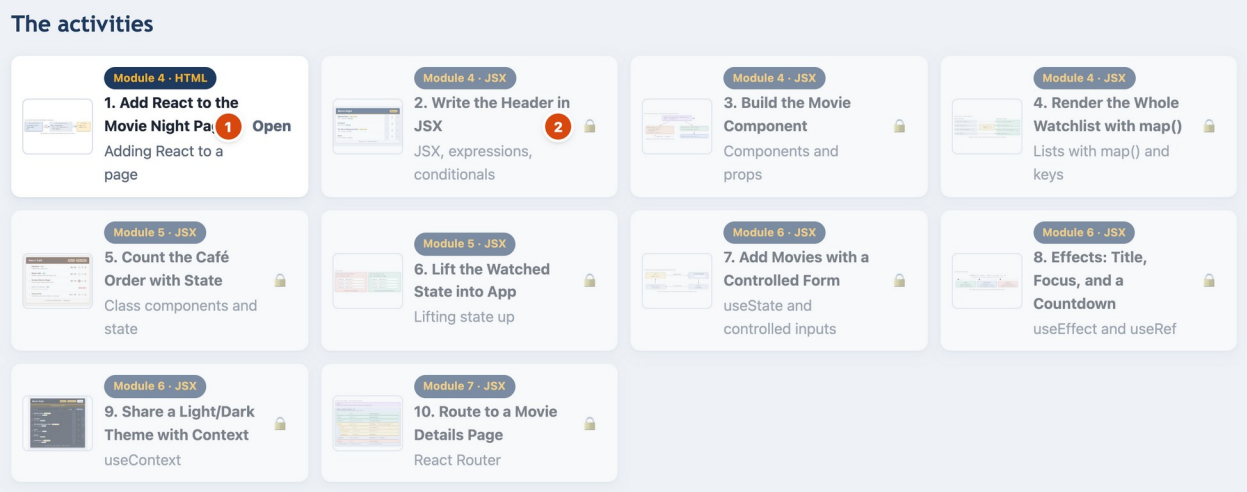


Figure 3 — Activity 1 is open (1); the rest are locked (2)

5. Inside an activity

Every activity uses the same screen: the task on the left, your editor and the running app on the right.

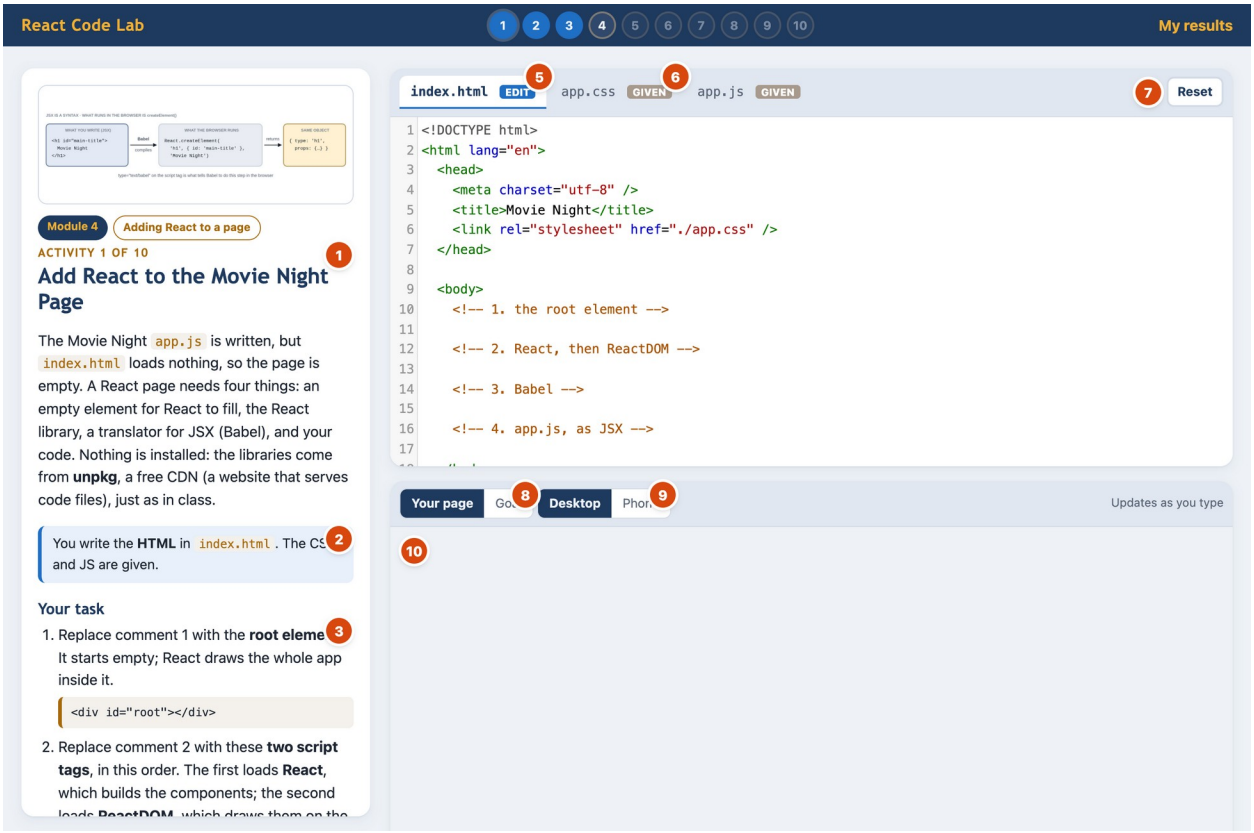


Figure 4 — an activity on a laptop screen; the numbers match the table below

#	What it is	What to do with it
1	Title and story	The module, the topic, and what the Movie Night or café app needs.
2	Which file you write	Activity 1: **HTML** in `index.html`. Activities 2 to 10: **JSX** in `app.js`.
3	Your task	Numbered steps. When you must type something exact, the step shows it in a grey code box.
—	What will be checked	Under the steps (scroll the left panel). Click it to see every check and its points: your checklist before you submit.
5	The file you edit	Marked **EDIT**. Click in it and type.
6	Given files	Marked **GIVEN** (`app.css`, and `app.js` or `index.html`). Read them; you cannot change them.
7	Reset	Puts the starter code back if you get lost (it asks first).
8	Your page / Goal	*Your page* runs your code; *Goal* shows what the finished app should look like.
9	Desktop / Phone	Shows the app at laptop width or phone width.
10	The preview	Your app, running. It updates half a second after you stop typing. Click its buttons and type in its inputs.
—	Submit	The blue button under the preview. It grades your code and shows your report.

The Console strip

Under the preview is a dark **Console** strip, just like the console in your browser's DevTools. `console.log(…)` output appears there. React's warnings appear with a △ (for example a missing `key`), and errors appear in red with a ⚠ and the **line number in your `app.js`**:

> ⚠ Uncaught ReferenceError: setTitle is not defined (app.js line 23)

If the preview is blank, the console nearly always says why.

The given files and the Goal

Click a **GIVEN** tab to read that file. `app.css` holds every class name the activity uses (`watchlist`, `movie`, `stats`, `toggle`…), and the given `app.js` or `index.html` shows how your part fits in. Click **Goal** at any time to see the finished app; it shows the result, not the code.

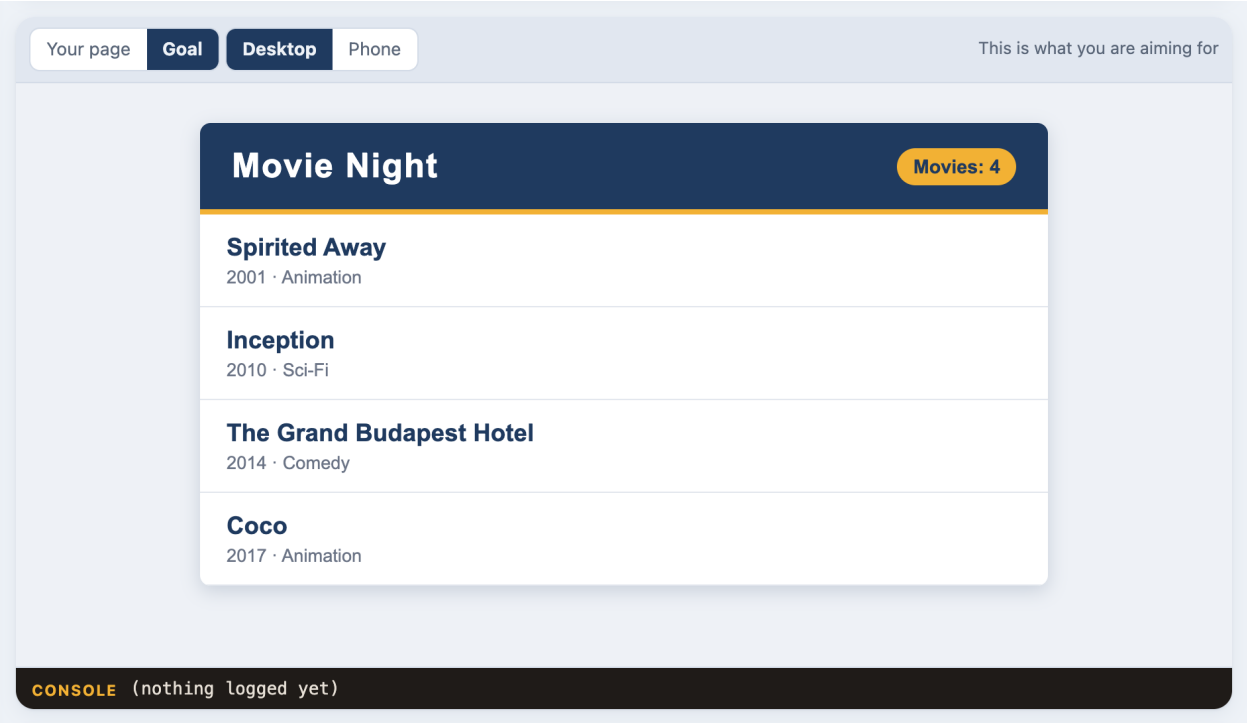


Figure 5 — the Goal view of Activity 1

6. Your first activity, step by step

Activity 1 is the only one where you write HTML: you load React into the Movie Night page, exactly as in our first CDN project.

1. On the home page, type your name and click **Start Activity 1**.
2. Read the story, then open **What will be checked**: six checks.
3. In `index.html`, each step tells you which comment to replace, why, and the exact lines (Figure 6, left). Replace the four comments with these lines, in this order:

```
<div id="root"></div>

<script crossorigin
src="https://unpkg.com/react@18.3.1/umd/react.development.js"></script>
<script crossorigin src="https://unpkg.com/react-dom@18.3.1/umd/react-
dom.development.js"></script>
<script src="https://unpkg.com/@babel/standalone@7/babel.min.js"></script>
<script type="text/babel" src="./app.js"></script>
```

Line	What it does
`<div id="root">`	An empty box. React draws the whole app inside it.
`react@18.3.1`	**React** itself: it builds components and keeps track of state.
`react-dom@18.3.1`	**ReactDOM**: it draws React's components on the web page. It needs React, so it comes second.
`@babel/standalone`	**Babel**: browsers cannot read JSX, so Babel translates it into plain JavaScript.
`type="text/babel" … app.js`	Your app. `type="text/babel"` tells Babel to translate this file before it runs.

4. Watch the preview: the Movie Night header and four movies appear (Figure 6, right). If the page stays empty, read the Console strip.
5. Click **Submit Activity 1**, read the report (Section 7), then click **Next: Activity 2 →**.

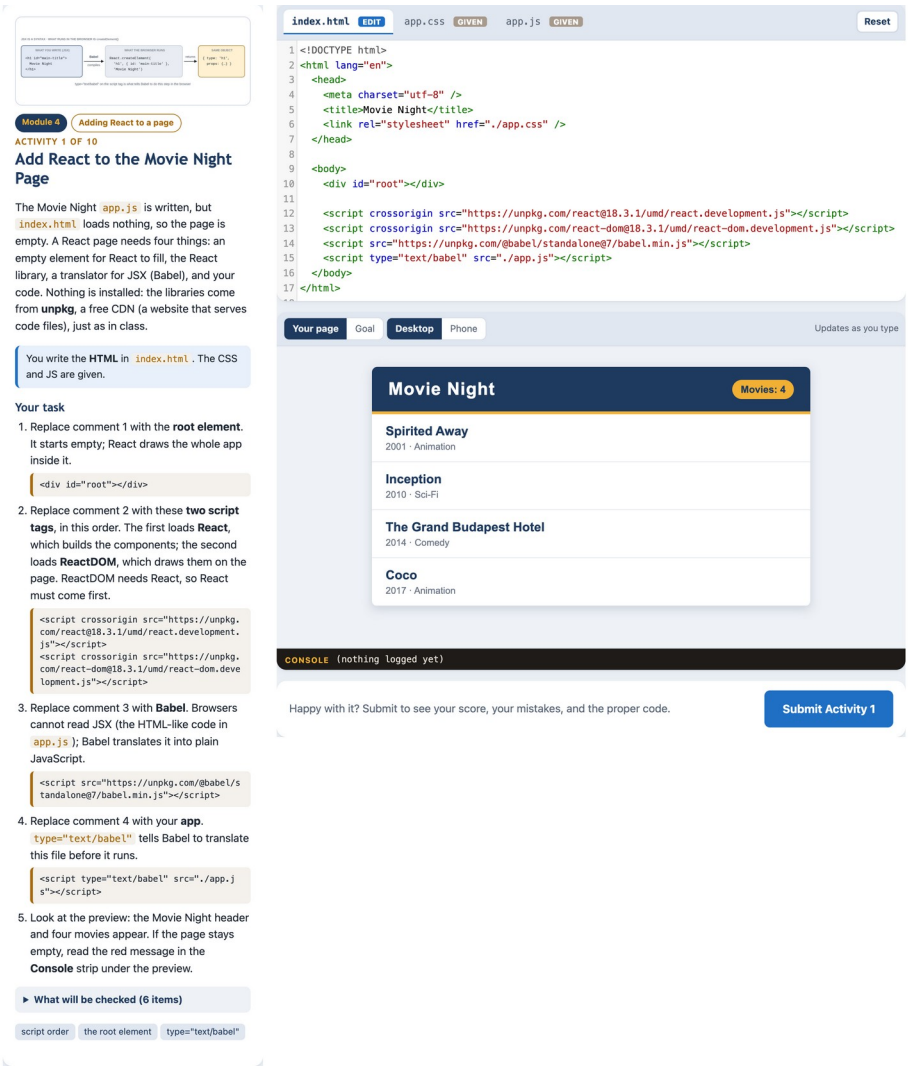


Figure 6 — Activity 1: the steps with the exact lines (left), and the finished page (right)

> **GOOD HABITS**
> Type the code rather than pasting it: that is how JSX sticks. Keep it indented. And when the preview breaks, look at the last thing you typed: a missing `}`, `)` or `/>` is the usual cause.

7. Reading your report

After you submit, the report appears under your work. It is the most useful part of the lab.

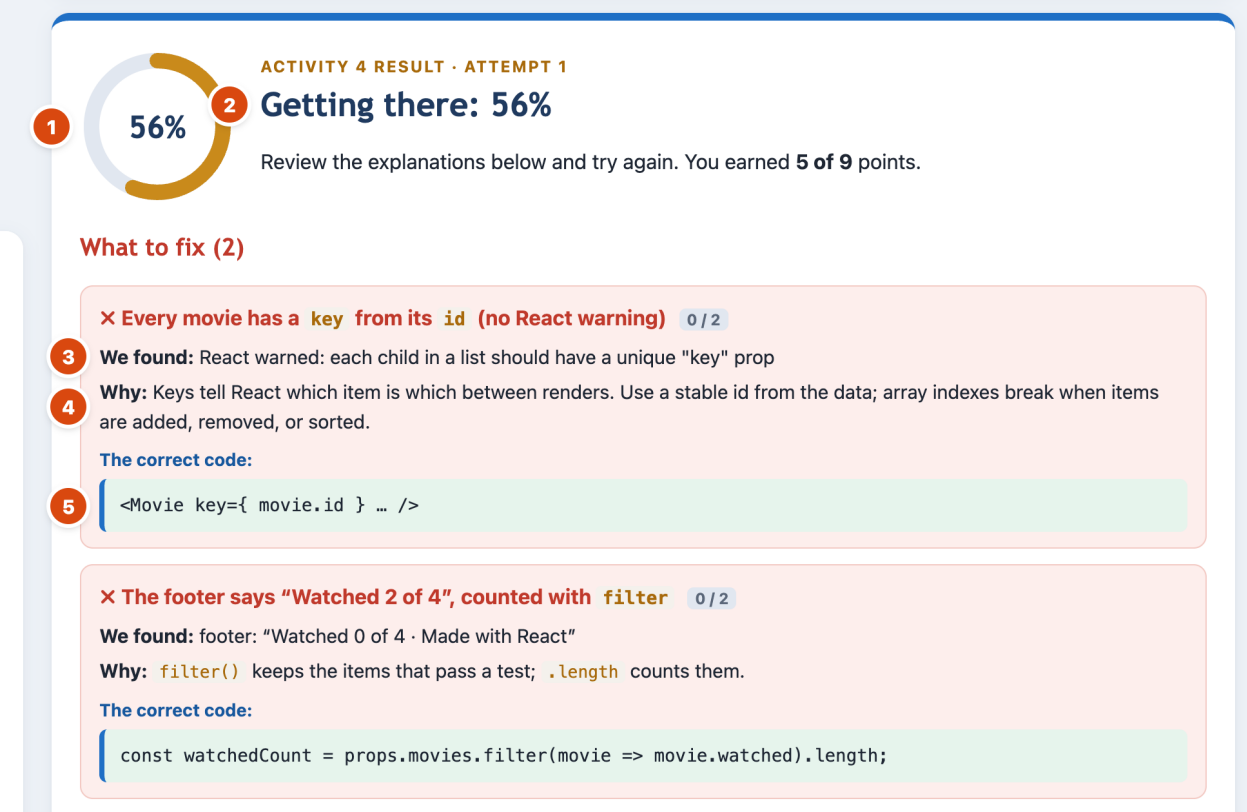


Figure 7 — the top of a report for Activity 4, with two mistakes; the numbers match the table below

#	What it is	What to do with it
1	Your score	The percentage of points you earned; its colour shows how you did.
2	Verdict	A one-line summary and your points, for example 8 of 15.
3	We found	What the lab saw when it ran your app, for example the React warning about a missing `key`.
4	Why	Why the rule matters, in plain words.
5	The correct code	Code that passes the check. Compare it with yours, then fix your own.

Score	Colour	Verdict	What to do
90–100%	Blue	Excellent	Move on.
75–89%	Green	Good work	Fix the small things, then move on.
50–74%	Amber	Getting there	Fix the mistakes and submit again.
0–49%	Red	Keep practising	Re-read the steps and the explanations, fix, and submit again.

Below the mistakes you may see **notes about your code** that are not scored, such as a syntax error with its line number or a React warning, and a list of the checks you passed.

The report then lines up **your file and the proper file** side by side:

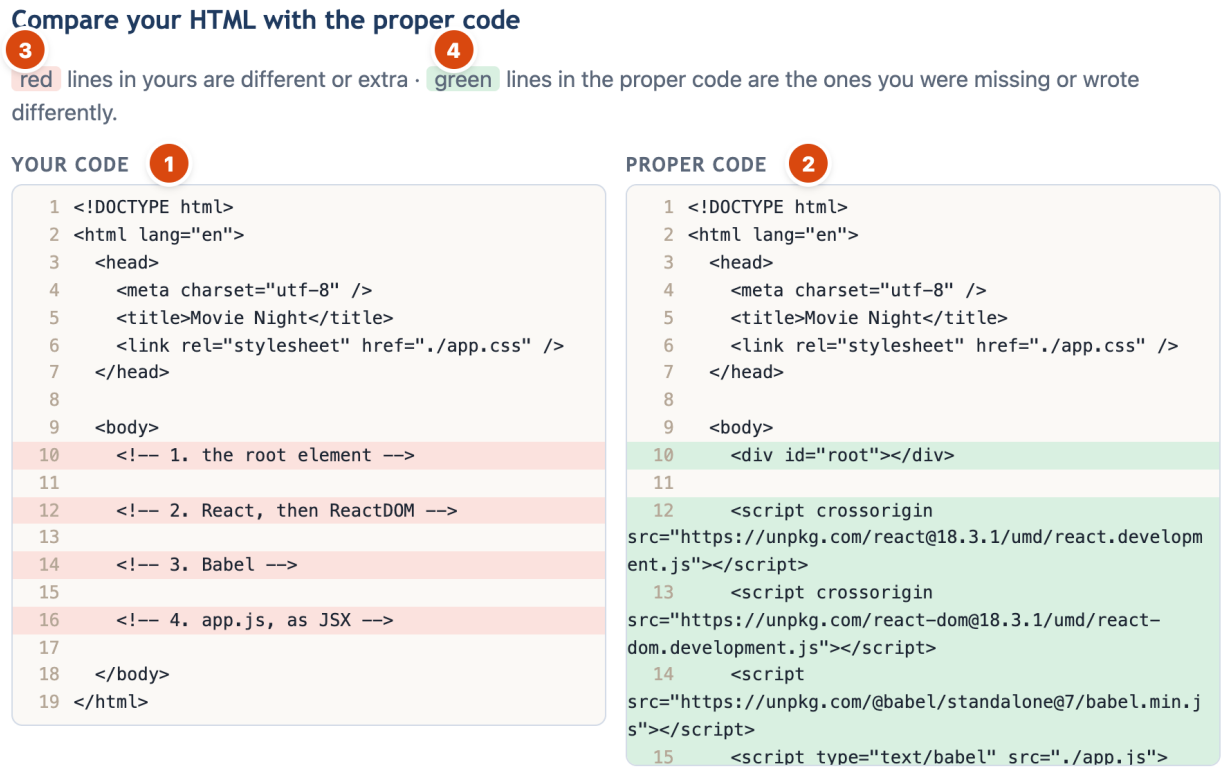


Figure 8 — your code (1) and the proper code (2); red lines (3) differ or are extra in yours, green lines (4) are what you were missing

Last, it shows **your app next to the goal**, with two buttons: **Try this activity again** (back to your code) and **Next: Activity N →**.

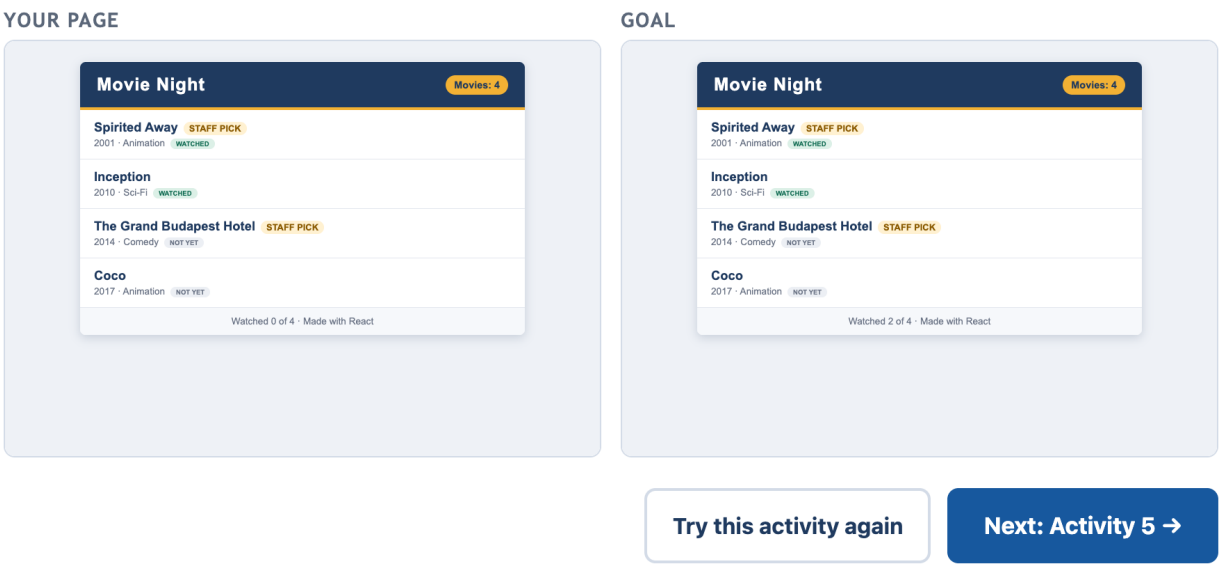


Figure 9 — your app and the goal side by side: the footer in yours still says “Watched 0 of 4”

8. Moving through the lab

The circles at the top show your progress at a glance:

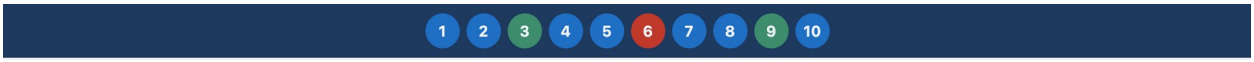


Figure 10 — the activity circles after a full run

Circle	Meaning
White border	The activity you are on now
Faded	Locked: submit the activity before it first
Blue / green	Submitted, 75% or more
Amber	Submitted, 50–74%
Red	Submitted, below 50%

Click any open circle to go back. Your last code is still there, with your last report under it. Submit again to improve the score.

---

9. Your final results

After Activity 10, click **See my final results →** (or **My results** at the top).

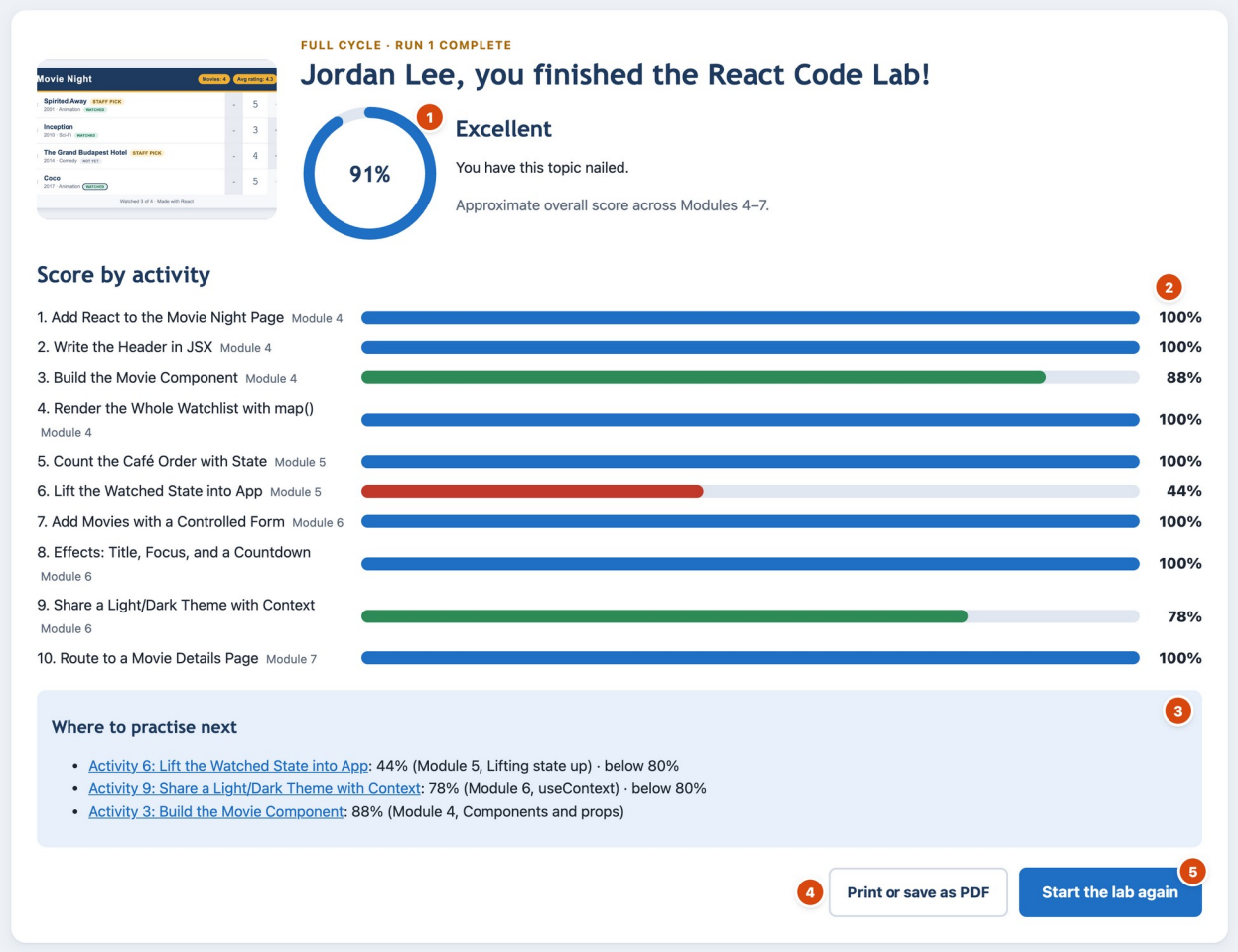


Figure 11 — the results page; the numbers match the table below

#	What it is	What to do with it
1	Overall score	The average of your ten activity scores.
2	Score by activity	One bar per activity; click a bar to go back to it.
3	Where to practise next	Your lowest-scoring activities, with links.
4	Print or save as PDF	Opens the print window: choose **Save as PDF** to keep or hand in your results.
5	Start the lab again	Saves this run in your history and starts a fresh run from Activity 1.

A second run starts every activity from its starter code again: good practice before a quiz. Your earlier runs and your best score are listed on the home page.

10. Tips for the React activities

- **`className`, not `class`.** JSX is JavaScript, and `class` is a JavaScript keyword.
- **One parent element.** A component returns one element; wrap siblings in a `<div>` or a fragment `<> … </>`.
- **Curly braces mean JavaScript.** `{ movies.length }` shows a value; `{ watched && <span>… </span> }` shows something only when `watched` is true.
- **Lists need keys.** `movies.map(movie => <Movie key={ movie.id } … />)`. A missing key shows a △ warning in the Console strip and costs points.
- **Never change state directly.** Use `setState(…)` or the setter from `useState`, and build new arrays and objects with `[...list, item]` and `{ ...movie, watched: true }`.
- **Pass functions, do not call them.** `onClick={ () => props.onToggle(props.id) }`, not `onClick={ props.onToggle(props.id) }` (that runs while rendering and crashes the app).
- **Effects with cleanups.** If an effect starts a timer, return a function that stops it.
- **Activity 10 uses `MemoryRouter`.** The preview has no address bar, so the router keeps the address in memory. In your Vite project you use `BrowserRouter` the same way, and `useParams()` still gives you the id as a **string**: convert it with `Number()` before comparing.
- **Use the steps' grey code boxes.** They show the exact shape to type; `…` marks the parts you fill in.

---

11. Troubleshooting

Problem	What to do
The preview is blank.	Read the Console strip. A red ⚠ gives the error and the line in `app.js`.
"ReactDOM is not defined" (Activity 1).	A script is missing, or ReactDOM comes before React. Check the order in Section 6.
"Unexpected token" with a line number.	A JSX syntax error: look for a missing `}`, `)`, `/>`, or a closing tag on or just before that line.
My progress disappeared.	You are in a different browser, on a different computer, or in a private window. Go back to the one you used.
An activity is locked.	Submit the activity before it.
I cannot type in the editor.	You are on a **GIVEN** tab. Click the tab marked **EDIT**.
My app looks right but I lost points.	Read *We found*: some checks look at how you wrote the code (`map()`, `key`, `useContext`, no hard-coded numbers).
The page stopped loading React.	Check your internet connection: the libraries come from unpkg.
I want to start an activity over.	Click **Reset** above the editor.

Still stuck? Bring a screenshot of your code and your report to class or office hours.